

A
MAJOR PROJECT REPORT

on

**Dr.Docker :A Large-Scale Security measurement of Docker
Image Ecosystem.**

Submitted in partial fulfillment of the requirements for the award of
the degree of Bachelor of Technology

by

Gandi Varshini
(22EG105B17)

Aenugu Manish Reddy
(22EG105B12)

Chirva Sai Koushik
(22EG105B10)

Kollawajjala Sreddutt
(22EG105B31)



Under the guidance of

Dr.K.Madhuri

Associate Professor

Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ANURAG UNIVERSITY
VENKATAPUR-500088
Year 2025-26

CERTIFICATE

This is to certify that the Project Report entitled “**Dr.Docker:A Large-Scale Security Measurement of Docker Image Ecosystem.**” being submitted by Ms. Gandhi Varshini, Mr. Aenugu Manish Reddy, Mr. Chirva Sai Koushik, Mr.kollawajjala Sreddutt bearing the Hallticket numbers 22EG105B17, 22EG105B12, 22EG105B10, 22EG105B31 in partial fulfillment of the requirements for the award of the degree of the Bachelor of Technology in Computer Science and Engineering to Anurag University is a record of bonafide work carried out by them under my guidance and supervision for the academic year 2025-2026.

The results presented in this report have been verified and found to be satisfactory. The results embodied in this report have not been submitted to any other University for the award of any other degree or diploma.

Signature of Supervisor

Dr. K. Madhuri

Associate Professor

Signature of Dean

Dr. G. Vishnu Murthy

Dean, CSE

EXTERNAL EXAMINER

ACKNOWLEDGMENT

We would like to express our sincere thanks and deep sense of gratitude to project supervisor **Dr. K. Madhuri** for her constant encouragement and inspiring guidance without which this project could not have been completed. Her critical reviews and constructive comments improved our grasp of the subject and steered to the fruitful completion of the work. Her patience, guidance and encouragement made this project possible.

We would like to express our sincere gratitude to **Dr. Archana Mantri**, Vice Chancellor, Anurag University for their encouragement and support.

We would like to express our special thanks to **Dr. V. Vijaya Kumar**, Dean School of Engineering, Anurag University, for their encouragement and timely support in our B. Tech program.

We would like to acknowledge our sincere gratitude for the support extended by **G. Vishnu Murthy**, Dean, Department of Computer Science Engineering, Anurag University. We also express my deep sense of gratitude to **Dr. V. Ramakrishna**, academic coordinator **Dr. G. Balram**, PQMC Chair, **Mrs. B. Ujwala**, Project Coordinator, Project Review and Project Quality & Monitoring Committee members, whose research expertise and commitment to the highest standards continuously motivated me during the crucial stage of our project work.

DECLARATION

We hereby declare that the Project Report entitled “**Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem.**” submitted for the award of Bachelor of Technology Degree is our original work. It has not been submitted to any other University or Institution for the award of any degree or diploma.

Date:

Gandi Varshini

Place:

(22EG105B17)

Aenugu Manish Reddy

(22EG105B12)

Chirva Sai Koushik

(22EG105B10)

Kollawajjala Sreddutt

(22EG105B31)

ABSTRACT

Docker has become one of the most widely used container technologies in cloud computing and DevOps environments because it allows applications to be packaged and deployed efficiently using container images. These images are widely shared through public repositories and reused by developers to build new applications. However, many publicly available Docker images contain security issues such as outdated software, known vulnerabilities, exposed secrets, and unsafe configurations.

When insecure images are reused as base images, these risks can spread to many other dependent images in the **Docker ecosystem**. Because of this reuse, security problems may silently propagate across applications without the knowledge of developers. Therefore, understanding the overall security condition of Docker images has become an important challenge.

This project, “**Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem**” focuses on analyzing Docker images on a large scale to study their security status. Docker images are collected from public repositories and analyzed using an automated framework that detects vulnerabilities, outdated dependencies, and configuration risks. The detected issues are converted into measurable security metrics such as vulnerability counts and severity levels.

The results are further analyzed to understand security patterns across the Docker image ecosystem. The final outcomes are presented through simple reports and visual representations that help users identify risky images and select safer alternatives before deployment. This approach supports developers, organizations, and researchers in understanding container security risks and encourages safer practices in modern cloud environments.

CONTENTS

S.NO		PAGE NO
1	Introduction	1
	1.1 Motivation	
	1.1.1 Importance of Containerization	
	1.1.2 Growth of Docker Ecosystem	
	1.1.3 Need for Security Analysis of Docker Images	
	1.2 Problem Definition	
	1.2.1 Security Risks in Docker Images	
	1.2.2 Limitations of Existing Research	
	1.2.3 Security Issues in Dependent Docker Images	
	1.3 Objectives of the Project	
	1.3.1 Docker Image Collection	
	1.3.2 Security Analysis of Images	
	1.3.3 Security Metric Calculation	
	1.3.4 Result Visualization and Reporting	
2	Literature Survey	
	2.1 Introduction	
	2.1.1 Container Security Survey: Exploits, Attacks, and Defenses (2025)	
	2.1.2 Research on Docker Container Network Isolation and Security Management for Multi-Tenant Environments (2023)	
	2.1.3 Empirical Study of the Docker Smells Impact on the Image Size (2024)	
	2.1.4 Malware Detection in Docker Containers: An Image is Worth a Thousand Logs (2025)	
	2.1.5 Risk Classification of Docker Images Using Machine Learning and Vulnerability Metrics (2025)	
	2.1.6 Large-Scale Security Measurement of Docker Ecosystem (2025)	
	2.2 Research Gap	

INTRODUCTION

Containerization has become an important technology in modern cloud computing and DevOps environments. It allows applications and their dependencies to be packaged together so that they can run consistently across different systems. Among various container technologies, Docker has gained widespread popularity because it provides a lightweight and efficient way to build, distribute, and deploy applications. Docker uses container images that contain the application code, libraries, and runtime environment required for execution. These images enable developers to quickly deploy applications without worrying about differences in system configurations.

Docker images are commonly shared through public repositories such as Docker Hub. Developers often reuse existing images as base images while building new applications. This practice makes software development faster and more efficient. However, the large-scale reuse of Docker images also introduces potential security risks. Many publicly available images contain outdated software packages, known vulnerabilities, exposed secrets, or insecure configurations. When such images are reused without proper security checks, these vulnerabilities may propagate to other images and applications.

Security issues in Docker images can significantly affect the reliability of container-based systems. In many cases, developers download and use images from public repositories without verifying their security status. As a result, insecure images may be deployed in real systems, creating opportunities for attackers to exploit vulnerabilities. Moreover, since Docker images often depend on other base images, security problems in one image can spread to many other images that rely on it.

To address this issue, it is important to study the overall security condition of the Docker image ecosystem. Understanding how vulnerabilities appear and propagate across Docker images can help developers and organizations make better decisions while selecting images for deployment. Therefore, a systematic and large-scale analysis of Docker images is necessary to identify security risks and evaluate the overall health of the ecosystem.

This project, “Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem,” focuses on analyzing Docker images on a large scale to understand their security characteristics. The project aims to detect vulnerabilities, analyze security patterns, and provide insights that can help users identify safer Docker images for deployment in cloud environments.

In recent years, container technologies have become a core component of modern cloud-native applications. Organizations adopt containerization because it simplifies application deployment and improves resource utilization. Containers are lightweight compared to traditional virtual machines since they share the host operating system while maintaining isolated environments for applications. This approach allows multiple containers to run efficiently on the same system without consuming excessive resources. As a result, container-based solutions are widely used in large-scale distributed systems and microservice architectures.

Docker has also contributed to improving the collaboration between development and operations teams. In traditional software development, developers often faced challenges when applications behaved differently across development and production environments. Docker addresses this issue by providing a consistent environment where applications can run the same way regardless of the underlying infrastructure. This consistency reduces deployment errors and makes it easier to maintain and update applications in complex software systems.

Despite these advantages, security remains a critical concern in container ecosystems. Because Docker images are frequently reused and shared across repositories, vulnerabilities present in one image can quickly affect many other images built on top of it. In addition, many developers may not have complete visibility into the internal components of the images they download. Therefore, studying the security characteristics of Docker images at a large scale is essential for identifying common risks and improving the overall security practices of container-based environments.

1.1 MOTIVATION

The rapid growth of cloud computing and DevOps practices has increased the use of container-based applications in modern software systems. Organizations rely heavily on container platforms to deploy applications quickly and efficiently. Public container repositories such as Docker Hub host a large number of Docker images that developers use as building blocks for their applications. This widespread sharing and reuse of images has made the Docker ecosystem very large and continuously growing.

Although this reuse improves development speed, it also introduces important security concerns. Many publicly available Docker images may contain outdated software packages, vulnerable libraries, or weak configurations. Since these images are created and maintained by different contributors, their security quality may vary significantly. Developers often download and use these images directly without performing a detailed security analysis, which may unintentionally introduce security risks into applications.

Another challenge arises from the dependency structure of Docker images. New Docker images are frequently built on top of existing base images. If the base image contains vulnerabilities, those weaknesses may propagate to several other images that depend on it. This situation makes it difficult to understand how security risks spread across the Docker ecosystem and increases the possibility of insecure containers being deployed in production environments.

Because of these concerns, analyzing the security condition of Docker images has become an important research problem. A systematic analysis of Docker images can help identify common vulnerabilities, understand security patterns, and evaluate the overall security status of the ecosystem. Motivated by this need, the Dr.Docker project focuses on studying Docker images at a large scale to measure their security characteristics and provide insights that help developers select safer images for deployment.

1.1.1 Growth of the Docker Ecosystem

In recent years, container technologies have become an important part of modern software development and deployment. Docker, in particular, has gained widespread adoption due to its simplicity and efficiency in packaging and running applications. Developers can easily create Docker images that contain all necessary components required to run an application. These images can be shared through public repositories such as Docker Hub, allowing developers around the world to reuse existing images while building new software systems. As a result, the Docker ecosystem has grown rapidly and now contains millions of publicly available container images.

1.1.2 Security Challenges in Public Docker Images

Although the large Docker ecosystem provides many benefits, it also introduces several security concerns. Many Docker images available in public repositories are created by different contributors and may not always follow secure development practices. Some images may include outdated software libraries, vulnerable dependencies, exposed credentials, or weak configurations. If such images are used directly in applications, they may introduce security vulnerabilities that could compromise the system. Since developers frequently download and reuse images from public repositories, the security quality of these images becomes an important factor for maintaining reliable container-based systems.

1.1.3 Need for Large-Scale Security Analysis

Another challenge arises from the dependency relationships between Docker images. Developers often build their applications on top of existing base images. If a base image contains vulnerabilities, these issues can propagate to many other dependent images. This makes it difficult to understand how security risks spread across the entire ecosystem. Therefore, a large-scale analysis of Docker images is necessary to identify common vulnerabilities and evaluate the overall security condition of the Docker environment. The **Dr. Docker project** is motivated by this need and aims to analyze Docker images systematically to provide insights that help developers choose safer images for deployment.

1.2 PROBLEM DEFINITION

The rapid adoption of container technologies has significantly increased the use of Docker images in cloud computing and DevOps environments. Developers frequently rely on publicly available Docker images to quickly build and deploy applications. These images provide preconfigured environments that simplify application development and reduce setup time. However, the convenience of using publicly shared images also introduces important security concerns.

Many Docker images available in public repositories may contain security weaknesses that are not immediately visible to users. These weaknesses can include vulnerable software packages, outdated dependencies, exposed configuration files, and insecure system settings. When developers download and use such images without carefully examining their internal components, these issues can become part of the deployed application environment. As a result, applications may unintentionally inherit security vulnerabilities from the images they are built upon.

Another difficulty arises from the dependency structure of Docker images. New container images are often created by extending existing base images. If a base image contains vulnerabilities or insecure components, these issues can spread to many other images that depend on it. This dependency chain makes it difficult to track how security risks propagate throughout the Docker ecosystem. Consequently, insecure images may continue to be reused in different projects without developers being aware of the potential risks.

Although several studies have explored container security, many of them focus on limited datasets or specific types of threats. This makes it challenging to obtain a clear understanding of the overall security condition of Docker images across the ecosystem. Therefore, a comprehensive analysis of Docker images is required to identify common vulnerabilities and evaluate their impact on container-based applications. Addressing this challenge is essential for improving the reliability and security of modern containerized systems.

1.2.1 Security Risks in Docker Images

Docker images obtained from public repositories may contain several types of security risks. These risks often arise due to outdated software packages, vulnerable libraries, exposed configuration files, or weak system settings included within the image. Since Docker images are frequently created and shared by different developers, the level of security maintenance can vary significantly. Some images may not receive regular updates, leaving known vulnerabilities unpatched. When such images are used to deploy applications, they can expose systems to potential attacks and reduce the reliability of container-based environments.

1.2.2 Limitations of Existing Research

Several research studies have attempted to analyze the security of Docker images, but many of them focus on limited datasets or specific types of security problems. For example, some studies concentrate mainly on vulnerability detection, while others focus on configuration issues or malware behavior individually. Because these studies analyze only a small number of images or address only one type of risk, they do not provide a complete understanding of the security condition of the Docker ecosystem. Therefore, broader analysis methods are required to examine security risks across a large number of Docker images.

1.2.3 Security Issues in Dependent Docker Images

Another challenge arises from the dependency relationships between Docker images. Developers commonly create new images by extending existing base images that provide the basic runtime environment. If a base image contains security weaknesses or outdated components, those issues can also appear in the images built on top of it. This dependency structure means that a single insecure base image can influence multiple dependent images within the ecosystem. Understanding these relationships is important for identifying how security risks spread and for improving the overall security of container-based applications.

1.3 OBJECTIVE OF THE PROJECT

The **Dr. Docker project** aims to analyze the security condition of Docker images used in container-based environments. As container technologies become widely used in cloud computing and DevOps practices, developers increasingly rely on publicly available Docker images from repositories such as Docker Hub to build and deploy applications. While this approach simplifies development and speeds up deployment, it can also introduce security risks when images are used without proper verification.

Many Docker images may contain outdated software packages, vulnerable dependencies, or insecure configurations that are not immediately visible to users. Because a large number of images are continuously uploaded to public repositories, manually checking their security status becomes difficult. Therefore, a systematic approach is required to analyze Docker images on a large scale and identify potential security risks within the ecosystem.

The Dr.Docker project addresses this challenge by collecting Docker images and analyzing them to detect vulnerabilities, dependency issues, and configuration problems. The project also generates security metrics and reports that help users identify safer images and avoid high-risk ones. By providing these insights, the project helps developers and organizations make better decisions when selecting Docker images and promotes safer practices in container-based software development.

In addition, the analysis performed by the project can contribute to improving awareness about security risks in container ecosystems. By highlighting common vulnerabilities and configuration weaknesses present in Docker images, the study encourages developers to follow safer image creation and usage practices. This can help reduce potential security threats in containerized applications and support the development of more secure cloud-based systems.

1.3.1 Docker Image Collection

The first objective of the project is to collect Docker images from public repositories such as Docker Hub. These images are selected from commonly used repositories to represent real-world container usage. By gathering a large set of images, the project can analyze security patterns across different applications and environments. This dataset forms the foundation for performing large-scale security analysis of Docker images.

1.3.2 Security Analysis of Docker Images

Another objective is to analyze each Docker image to identify potential security issues. The analysis focuses on detecting vulnerabilities, outdated software components, and configuration weaknesses present within the image. This process helps determine whether the Docker image is suitable for use in container-based applications. Identifying these issues is important for improving the overall security of deployed containers.

1.3.3 Security Metric Calculation

After identifying security issues in Docker images, the project calculates security metrics that represent the level of risk associated with each image. These metrics may include vulnerability counts and severity levels that indicate how critical the detected issues are. By converting security information into measurable values, it becomes easier to compare different Docker images and evaluate their security status.

1.3.4 Result Visualization and Reporting

The final objective is to present the analysis results in a clear and understandable form. The project generates simple reports using tables or graphical representations that highlight high-risk and low-risk Docker images. These reports help developers and organizations make informed decisions when selecting Docker images for their applications. Clear reporting also improves awareness of security issues in container-based environments.

2.LITERATURE SURVEY

2.1 INTRODUCTION

Modern software development increasingly relies on container technologies to simplify application deployment and management. Containers provide lightweight environments that allow applications and their dependencies to run consistently across different systems without compatibility issues. This approach improves portability, scalability, and efficiency in modern software systems. Among the available container platforms, Docker has become one of the most widely adopted technologies because it allows developers to easily build, distribute, and deploy applications using container images.

Docker images act as reusable software packages that contain application code, libraries, runtime environments, and configuration files required for execution. These images are commonly stored and shared through public repositories such as Docker Hub, where developers can access a large number of pre-built images for various applications. By reusing existing images, developers can significantly reduce development time and simplify the process of application deployment across different environments. This widespread sharing and reuse of Docker images has contributed to the rapid growth of the container ecosystem.

However, the extensive use of publicly available Docker images also introduces several security concerns. Some images available in public repositories may contain outdated software components, vulnerable dependencies, insecure configurations, or hidden malicious content. When such images are used as base images for building new containers, the vulnerabilities present in them may propagate to other dependent images and applications. As a result, security risks can spread across multiple systems within the container ecosystem.

Researchers have conducted several studies to analyze security challenges associated with container technologies and Docker images. These studies focus on areas such as vulnerability detection, malware identification in containers, Docker image configuration analysis, and network isolation in multi-tenant environments. Reviewing

this research helps in understanding existing solutions and identifying security risks .

2.1.1 A Container Security Survey: Exploits, Attacks, and Defenses (2025)

Container technologies have become an essential part of modern software deployment because they provide lightweight and efficient virtualization compared to traditional virtual machines. However, the rapid growth of containerized applications has also introduced several security challenges. The research paper **“A Container Security Survey: Exploits, Attacks, and Defenses” (2025)** presents a comprehensive analysis of security issues associated with container environments. The study focuses on understanding different attack techniques, vulnerabilities, and defensive strategies used to protect container-based systems.

The authors analyze various types of attacks that can affect container platforms, including container escape attacks, privilege escalation, malware injection, and configuration-based vulnerabilities. These attacks can compromise container isolation and allow attackers to gain unauthorized access to host systems or other containers. The study highlights that containers share the host operating system kernel, which increases the potential risk of security breaches if proper protection mechanisms are not implemented.

To better understand container security risks, the researchers examined a large number of vulnerabilities and classified them into different exploit categories and attack vectors. The paper presents a detailed taxonomy that helps identify how security threats can occur at different layers of container architecture, including the application layer, container runtime, host system, and hardware level. This classification helps security researchers and developers understand the complexity of container-based threats.

The study also discusses several defense mechanisms that can improve container security. These include secure configuration practices, vulnerability scanning, runtime monitoring, network isolation, and access control policies. The authors emphasize that combining multiple security techniques is essential to build a robust container security framework. The insights provided by this research help developers and organizations understand common container security threats and adopt better protection strategies for containerized applications.

2.1.2 Research on Docker Container Network Isolation and Security Management for Multi-Tenant Environments (2023)

Docker containers are widely used in modern computing environments because they provide efficient application deployment and resource utilization. However, when multiple applications from different users run on the same infrastructure, security becomes a major concern. The research paper **“Research on Docker Container Network Isolation and Security Management for Multi-Tenant Environments” (2023)** focuses on improving security in container-based systems where multiple tenants share the same computing resources.

The study explains that in multi-tenant environments, several containers belonging to different users may operate on the same physical host. Without proper security mechanisms, containers may interfere with each other or gain unauthorized access to shared resources. Network communication between containers must therefore be carefully controlled to prevent data leakage, unauthorized access, and other potential security threats.

To address these issues, the authors propose a security framework that focuses on container network isolation and security policy management. The proposed approach introduces mechanisms that separate network communication between containers belonging to different tenants. By implementing controlled network access policies and secure communication channels, the system can protect sensitive data and maintain isolation between containers.

The research also evaluates the effectiveness of the proposed security mechanisms through experimental analysis. The results demonstrate that improved network isolation techniques can significantly enhance container security while maintaining efficient system performance. This study highlights the importance of secure network management in container environments and provides valuable insights for designing secure multi-tenant container platforms.

2.1.3 Empirical Study of the Docker Smells Impact on the Image Size (2024)

Docker images are created using configuration files known as Dockerfiles, which define the steps required to build container images. However, improper configuration practices in Dockerfiles can lead to inefficiencies and potential security risks. The research paper **“Empirical Study of the Docker Smells Impact on the Image Size” (2024)** investigates common issues in Dockerfiles known as *Docker smells* and analyzes how these issues affect Docker image quality and size.

Docker smells refer to poor configuration practices or deviations from recommended Dockerfile guidelines. These issues may include unnecessary packages, inefficient commands, or improper dependency management during the image creation process. Such practices can increase the size of Docker images and introduce additional components that may expand the attack surface of containerized applications.

In this study, the authors analyzed thousands of Dockerfiles collected from open-source repositories to identify common Docker smell patterns. They examined how these issues impact Docker image size and system efficiency. The research found that many Docker images contain avoidable configuration problems that increase image size and reduce overall efficiency. By correcting these configuration issues, developers can significantly reduce image size and improve container performance.

The findings of this research highlight the importance of following best practices when creating Docker images. Proper Dockerfile configuration not only improves performance but also helps reduce unnecessary dependencies that may introduce security vulnerabilities. This study provides valuable guidance for developers to build more efficient and secure Docker images within container-based environments.

2.1.4 Malware Detection in Docker Containers: An Image is Worth a Thousand Logs (2025)

The rapid adoption of container technologies has increased the importance of detecting security threats within container environments. Docker containers are widely used for deploying applications because they provide lightweight and portable execution environments. However, attackers may inject malicious software into containers, which can compromise the security of applications and underlying systems.

The research paper “**Malware Detection in Docker Containers: An Image is Worth a Thousand Logs**” (2025) investigates methods for identifying malware within containerized environments.

Traditional malware detection methods often rely on signature-based techniques that compare files with known malware patterns. While effective for known threats, these techniques struggle to detect new or modified malware variants. The study highlights the limitations of traditional approaches and emphasizes the need for advanced detection techniques capable of identifying hidden or unknown malicious behavior within containers.

The researchers propose a novel approach that analyzes the entire container file system instead of examining individual files. In their method, container data is transformed into image representations that can be analyzed to identify patterns associated with malicious activity. By examining the overall structure of container data, the system can detect suspicious modifications caused by malware.

Experimental results presented in the study demonstrate that analyzing container file systems can significantly improve malware detection accuracy. The proposed approach helps identify compromised containers more effectively than traditional techniques. This research contributes to improving container security by providing new methods for detecting malicious activity within Docker environments.

2.1.5 Risk Classification of Docker Container Images Using Machine Learning and Vulnerability Metrics (2025)

As the use of container images continues to grow, ensuring their security has become a major concern for developers and organizations. Docker images obtained from public repositories may contain various vulnerabilities that can affect the security of containerized applications. The research paper **“Risk Classification of Docker Container Images Using Machine Learning and Vulnerability Metrics” (2025)** focuses on identifying and classifying security risks associated with Docker images.

The study explains that traditional vulnerability scanning tools can identify security weaknesses but often generate large reports that are difficult to analyze and prioritize. As a result, developers may find it challenging to determine which vulnerabilities pose the greatest risk. The research proposes a system that uses vulnerability metrics to evaluate the security condition of Docker images.

In this approach, the researchers analyze Docker images using vulnerability scanning tools and collect various security-related features such as vulnerability counts, severity levels, and package characteristics. These features are then used to classify container images into different risk categories. This classification helps developers understand whether a particular image poses a high or low security risk.

The results of the study show that analyzing vulnerability metrics can help improve the process of identifying risky Docker images. By classifying images based on their security characteristics, developers can make better decisions when selecting container images for deployment. The research highlights the importance of vulnerability analysis for improving the security of container ecosystems.

2.1.6 Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem (2025)

The increasing popularity of Docker containers has led to the rapid growth of publicly available container images. These images are frequently reused by developers to build and deploy applications. However, the widespread reuse of container images also increases the possibility of security risks within the Docker ecosystem.

The research paper “**Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem**” (2025) examines the security conditions of Docker images on a large scale. The study analyzes Docker images collected from public repositories in order to identify various security threats present within the container ecosystem.

The researchers investigate several types of risks, including software vulnerabilities, secret leaks, misconfigurations, and malicious components embedded within Docker images. These issues can affect not only individual images but also other images that depend on them.

To perform large-scale analysis, the researchers developed a security analysis framework capable of examining thousands of Docker images. The framework analyzes image metadata, dependencies, and configuration information to detect security weaknesses. The study also explores how security risks can propagate through image dependency relationships within the Docker ecosystem.

The findings of this research reveal that a significant number of Docker images contain security vulnerabilities or configuration issues. The study emphasizes the importance of analyzing Docker images before using them in production environments. The results highlight the need for improved security practices and systematic analysis methods to maintain the security of the container ecosystem.

2.2 RESEARCH GAP

The research papers reviewed in the literature survey provide important insights into different security challenges associated with container technologies and Docker environments. The study on **container security exploits, attacks, and defenses** highlights the various threats that can affect containerized systems, including container escape attacks, privilege escalation, and configuration vulnerabilities. This research provides a broad understanding of security risks in container platforms and discusses several defense mechanisms to improve container security.

Another study focuses on **network isolation and security management in multi-tenant Docker environments**, where multiple containers belonging to different users share the same infrastructure. This research emphasizes the importance of secure communication and proper network isolation mechanisms to prevent unauthorized access between containers. The findings show that improving network isolation can significantly enhance the security of container-based systems operating in shared environments.

The research on **Docker smells and configuration issues** analyzes common mistakes present in Dockerfiles that can increase image size and introduce unnecessary dependencies. These configuration problems can reduce efficiency and potentially increase the attack surface of container images. The study highlights the importance of following best practices when building Docker images in order to create more efficient and secure containers.

Other research studies have focused on **malware detection in Docker containers** and **risk classification of container images using vulnerability metrics**. These works attempt to identify malicious software within container environments and analyze vulnerabilities present in Docker images. Such studies contribute to improving container security by developing techniques that can detect compromised containers and classify images based on their security risks.

Although these studies address important aspects of container security, most of them focus on specific problems such as malware detection, vulnerability classification, configuration issues, or network isolation. They generally analyze individual security threats rather than examining the overall security condition of the Docker image ecosystem.

As the number of publicly available Docker images continues to grow rapidly, the reuse of container images has increased significantly, which makes it difficult to understand the broader security risks present across the entire ecosystem.

Therefore, there is a need for a systematic and large-scale analysis of Docker images to evaluate their security characteristics and identify potential vulnerabilities that may affect container-based systems. Addressing this gap will help developers and organizations better understand the security risks associated with publicly available container images and support the development of more secure container environments.

3.PROPOSED SYSTEM

3.1 PROPOSED SYSTEM INTRODUCTION

To understand the security condition of the Docker image ecosystem, this project proposes a systematic framework called **Dr. Docker**, which performs large-scale analysis of Docker images available in public container registries. Docker images are widely used in modern cloud and DevOps environments because they allow applications and their dependencies to be packaged and shared easily. However, many publicly available images may contain hidden security risks such as software vulnerabilities, leaked secrets, unsafe configurations, or malicious components. These issues often remain unnoticed and may spread to other images because Docker images are frequently reused as base images.

The proposed system aims to analyze Docker images at a large scale and identify potential security threats before the images are deployed in real environments. The framework collects image information from Docker Hub, analyzes relationships between images through dependency analysis, and scans the images for different types of security threats. By performing this automated analysis, the system helps in measuring the overall security condition of the Docker image ecosystem and highlights risky images that may introduce vulnerabilities into cloud applications.

The proposed framework follows a structured methodology that includes data collection, dependency graph construction, threat detection, and security metric generation. These steps help in identifying vulnerabilities, secret leaks, misconfigurations, and malicious files present in Docker images. The detected issues are then converted into security metrics, which allow different images to be compared based on their risk levels.

By combining large-scale data collection with automated threat detection techniques, the proposed system provides a systematic approach for evaluating Docker image security. The results generated by this framework can assist developers and organizations in selecting safer container images and adopting better security practices when deploying applications in containerized environments.

3.2 PROPOSED SYSTEM ARCHITECTURE

The proposed system architecture of **Dr. Docker** is designed to perform large-scale security analysis of Docker images available in public container registries. The architecture follows a modular workflow in which different components perform specific tasks such as collecting image information, analyzing dependencies, detecting security threats, generating security metrics, and presenting the results. This structured design allows the system to analyze a large number of Docker images efficiently and helps in understanding the overall security condition of the Docker image ecosystem.

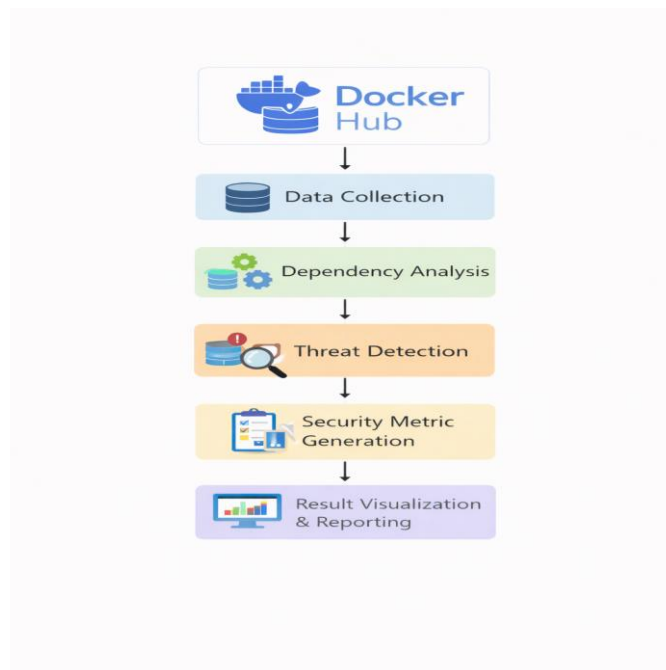


Figure: Proposed System Architecture of Dr. Docker

The architecture begins with **Docker Hub**, which acts as the primary source of Docker images. Docker Hub is the largest public container registry where developers upload and share container images. These images are widely reused in cloud and DevOps environments. Since many images are built using existing images as base images, security problems present in one image may propagate to several other images that depend on it. Therefore, it becomes important to analyze these images before they are deployed in real systems.

The first component in the architecture is the **Data Collection Module**. This module is responsible for collecting information about Docker image repositories from Docker Hub. The system gathers repository descriptions, image metadata, layer information, and other related details using automated crawling techniques. These descriptions contain useful information such as repository names, pull counts, image tags, and instructions used to build the image layers. Collecting this information helps in understanding how Docker images are distributed across the ecosystem and identifies images that are widely used by developers.

After collecting the repository information, the system performs analysis using the **Dependency Analysis Module**. Docker images are built using layered architectures where each image may depend on another image called the parent image. When developers create new images using existing base images, the new image inherits software packages, configurations, and files from the parent image. This creates a dependency relationship between images. The dependency analysis module studies these relationships and constructs a dependency structure that represents how images are connected within the ecosystem. This analysis helps identify critical images because security issues present in base images may propagate to many downstream images.

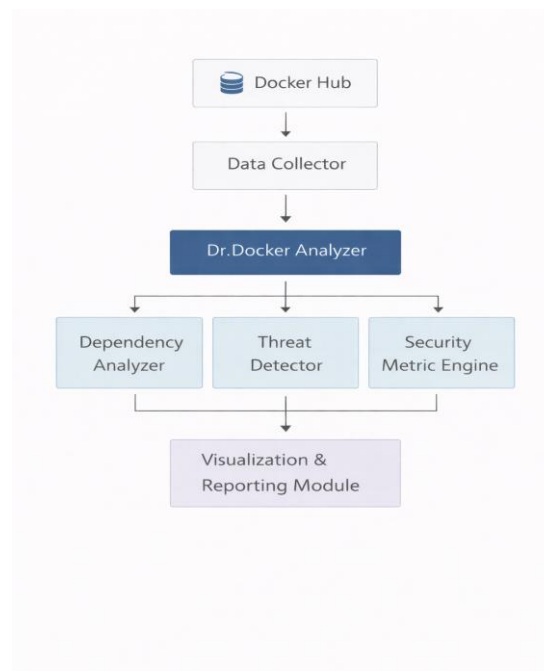


Figure: Component Architecture of the Dr. Docker System

The next component is the **Threat Detection Module**, which performs security analysis of the selected Docker images. This module scans the images to detect different types of security threats that may exist within the container environment. The system examines multiple aspects of the image, including repository descriptions, configuration metadata, and image contents. During this process, the module identifies potential security issues such as software vulnerabilities in installed packages, leaked secrets like API keys or tokens, insecure configuration settings, malicious files, and unsafe command parameters that may weaken container isolation. Detecting these threats at the image level helps prevent insecure images from being used in production environments.

Once the security threats are detected, the information is processed by the **Security Metric Generation** module. This module converts the detected security issues into measurable values that represent the security condition of each Docker image. For example, the system calculates the number of vulnerabilities present in the image, classifies them based on severity levels, and generates a risk score for each image. These security metrics help compare different Docker images and identify which images pose higher security risks within the ecosystem.

Finally, the results generated by the system are presented through the **Result Visualization and Reporting** module. This module organizes the analysis results in a clear and understandable format using tables, charts, or graphical summaries. These visual representations allow developers and researchers to easily identify high-risk images and understand security trends across the Docker ecosystem. By presenting the results in a structured way, the system helps users make informed decisions when selecting Docker images for deployment.

Overall, the proposed system architecture provides a systematic framework for analyzing the security of Docker images at large scale. By integrating data collection, dependency analysis, threat detection, and security evaluation processes, the system enables comprehensive measurement of security risks within the Docker image ecosystem and supports safer usage of container images in cloud environments.

3.3 UML DIAGRAMS OF THE PROPOSED SYSTEM

CLASS DIAGRAM

The class diagram represents the structure of the **Dr. Docker system** and shows the main components used for Docker image security analysis. The **Image Collector** gathers Docker images and metadata from Docker Hub, which are then stored for further processing.

The **Vulnerability Scanner** and **Secret Scanner** analyze the collected images to detect software vulnerabilities and exposed sensitive information such as API keys or tokens. These modules help identify potential security risks within Docker images.

The **Risk Engine** calculates the overall risk score based on detected issues, while the **PolicyEngine** evaluates deployment policies. Finally, the **Deployment Manager** manages container deployment and the **MTLS Manager** ensures secure communication between system components.

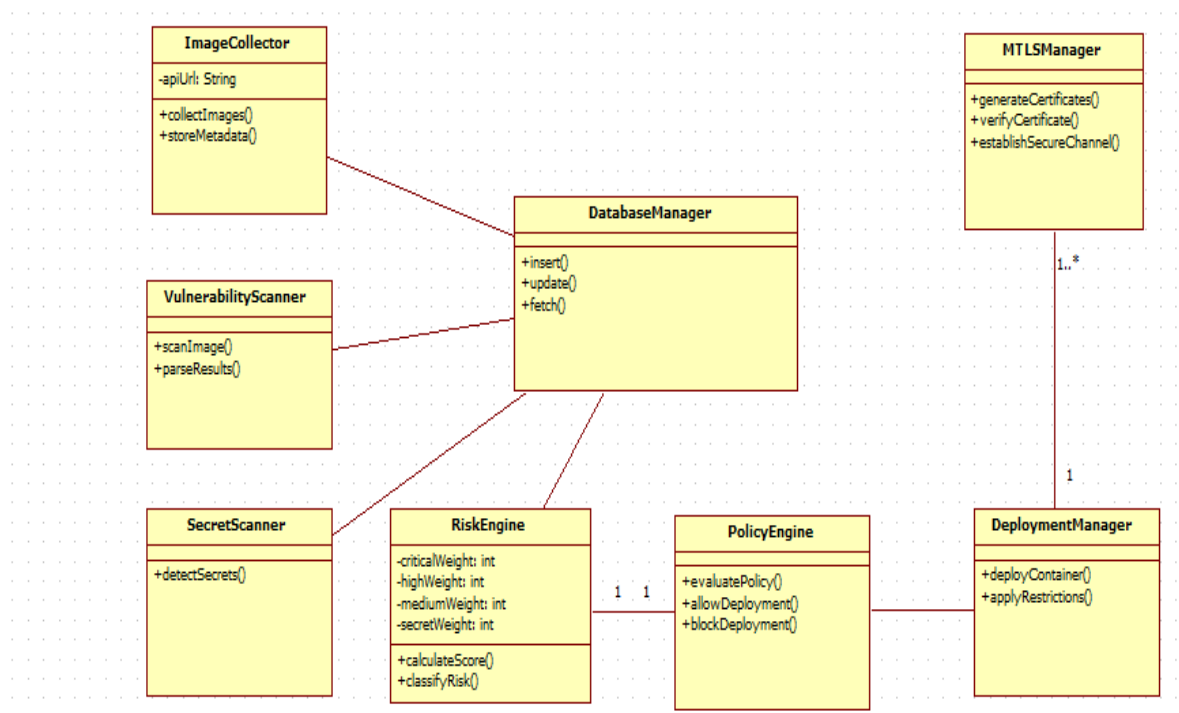


Figure :class diagram of the proposed system

ACTIVITY DIAGRAM

The activity diagram illustrates the workflow of the Dr.Docker system for analyzing Docker images and detecting security risks. The process begins by collecting Docker images and metadata from Docker Hub through the data collection module.

After collecting the images, the system performs security checks such as vulnerability scanning and secret detection. It analyzes image contents and configurations to identify threats like exposed credentials or vulnerable packages.

Finally, the system calculates the risk score of the image and determines whether the container can be safely deployed or should be blocked, ensuring secure container deployment.

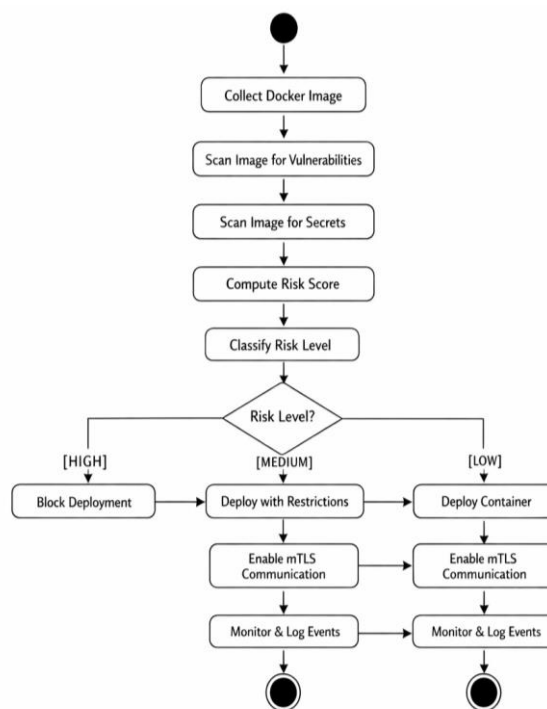


Figure :Activity diagram of the proposed System.

USECASE DIAGRAM

The use case diagram shows how users and external systems interact with the Dr.Docker system to perform Docker image security analysis. The main actors include the DevOps Engineer, System Administrator, Docker Hub API, and Container Runtime.

The DevOps Engineer submits Docker images for analysis and views security reports, while the System Administrator manages system policies and monitors security status before deployment.

The system retrieves images from Docker Hub and analyzes them for security risks. Based on the results, the container is either allowed for deployment or blocked to ensure secure container usage.

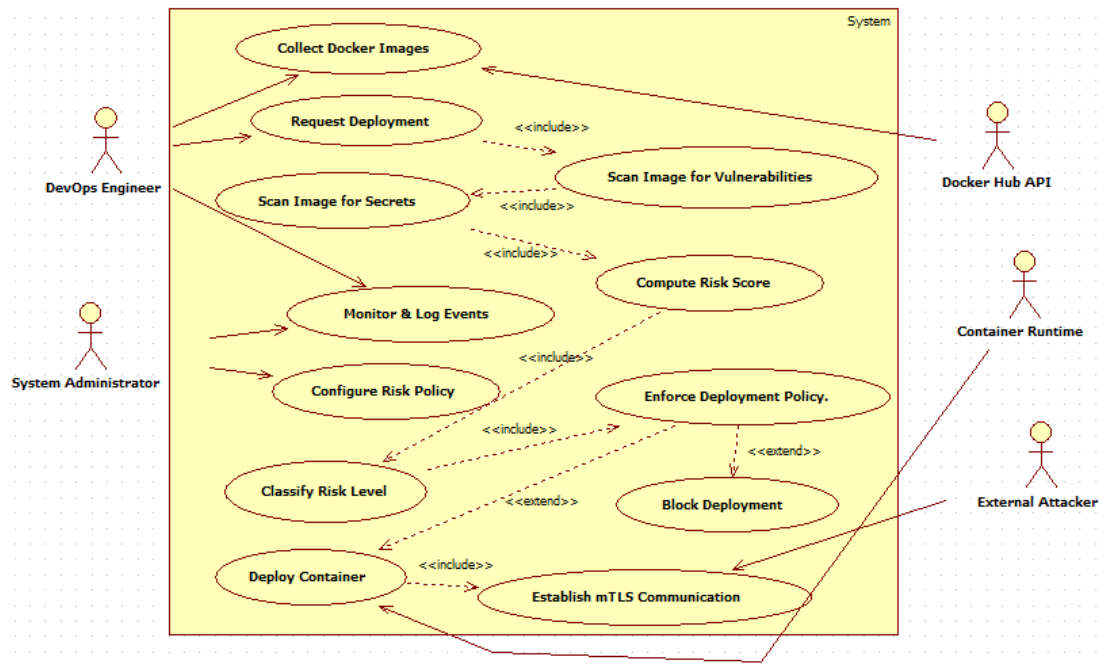


Figure : Use Case Diagram of proposed System.

SEQUENCE DIAGRAM

The sequence diagram illustrates the interaction between different components of the Dr. Docker system during the Docker image security analysis process. It shows the order in which system modules communicate with each other to perform image scanning and risk evaluation. The process begins when the user submits a Docker image for analysis. The system retrieves the image information from Docker Hub and sends it to the scanning modules. The vulnerability scanner and secret scanner analyze the image to detect security issues and generate scan results. After the scanning process, the Risk Engine calculates the overall risk score of the Docker image. The Policy Engine then evaluates deployment policies based on the calculated risk level and decides whether the container should be deployed or blocked to ensure secure deployment.

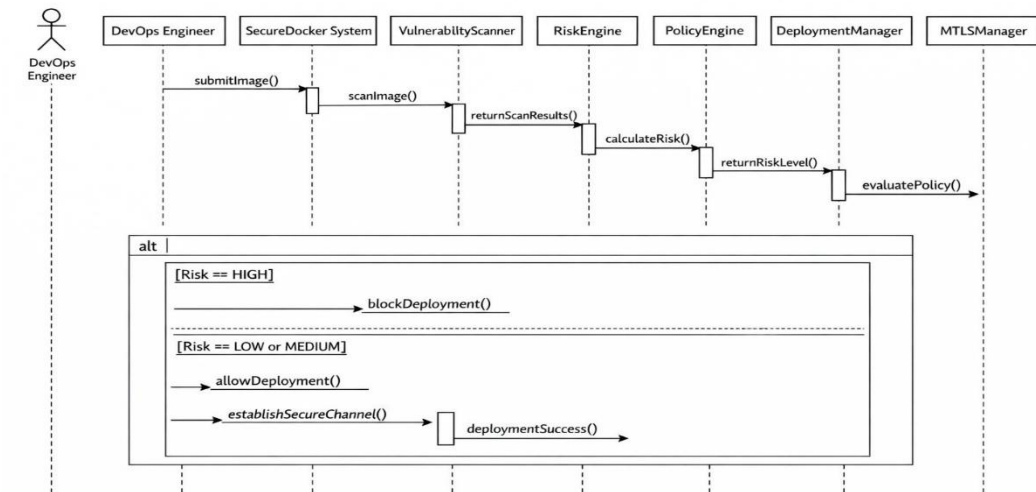


Figure: Sequence Diagram of Proposed System.

COMPONENT DIAGRAM

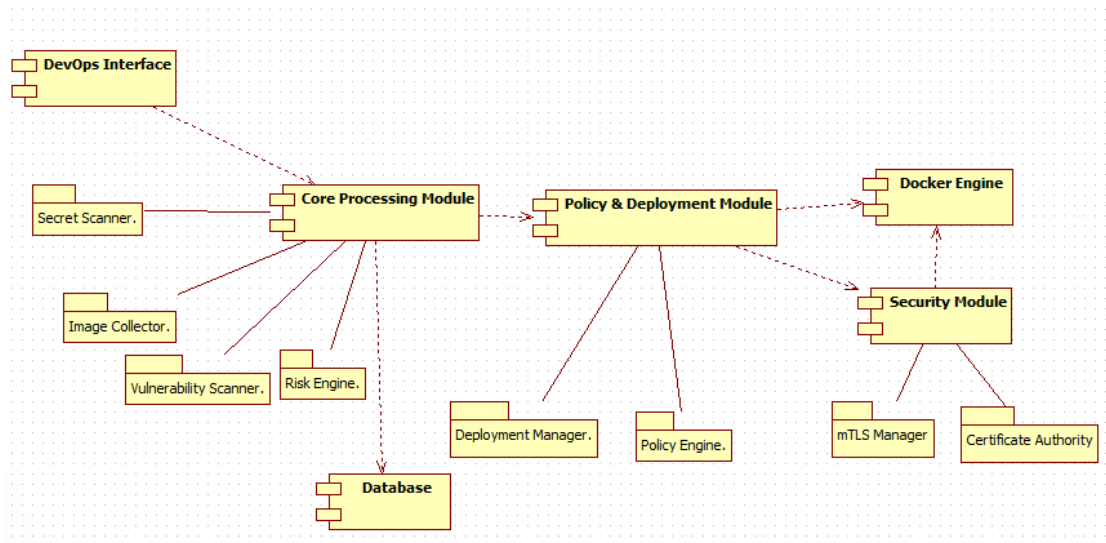


Figure: Component Diagram of proposed System.

The component diagram shows the overall structure of the Dr.Docker system and how different modules interact to perform Docker image security analysis. The DevOps Interface allows users to submit images, while the Core Processing Module manages communication between the main system components.

Modules such as the Image Collector, Vulnerability Scanner, Secret Scanner, and Risk Engine work together to collect Docker image data, detect vulnerabilities and exposed secrets, and calculate the risk level of the image. The results are stored in the Database for further processing.

The Policy and Deployment Module evaluates security policies and manages container deployment through the Deployment Manager and Policy Engine. The Security Module, including the mTLS Manager and Certificate Authority, ensures secure communication with the Docker Engine during deployment.

DEPLOYMENT DIAGRAM

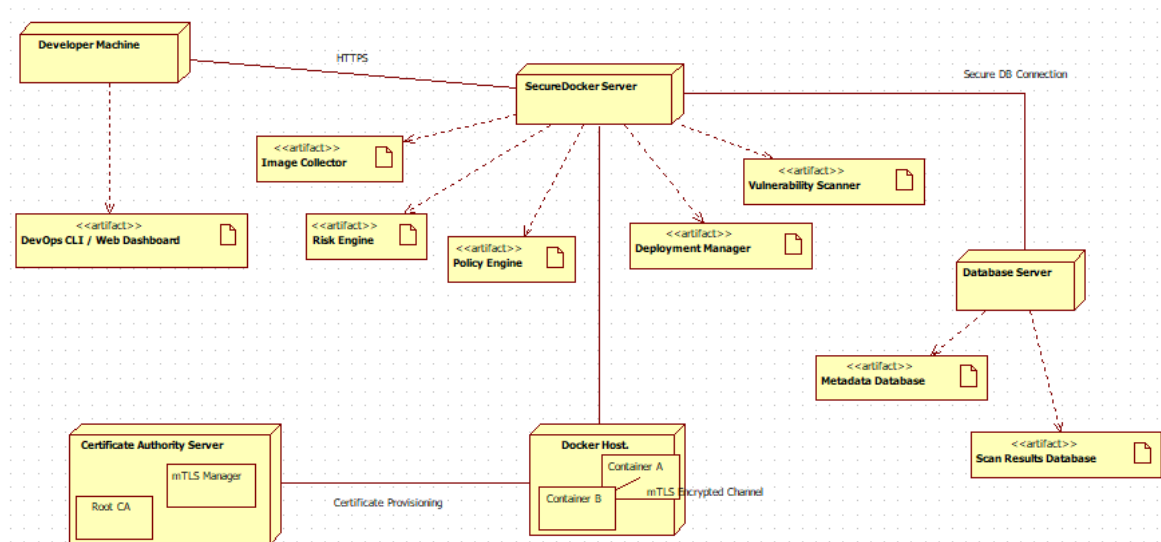


Figure: Deployment Diagram of proposed System.

The deployment diagram shows how the Dr.Docker system components are distributed across different hardware nodes. It includes the developer machine, secure Docker server, database server, and Docker host used for container deployment.

The Developer Machine connects to the Secure Docker Server through a secure connection. The server hosts key modules such as the Image Collector, Vulnerability Scanner, Risk Engine, Policy Engine, and Deployment Manager that perform Docker image security analysis. The Database Server stores metadata and scan results, while the Certificate Authority Server enables secure communication using mTLS authentication. Finally, the validated containers are deployed on the Docker Host, ensuring secure container execution.

3.4 DATA COLLECTION METHODOLOGY

The data collection methodology describes how the **Dr. Docker system** gathers **Docker image data for security analysis**. The primary source of data is **Docker Hub**, which is the largest public container registry used by developers to store and share container images. The system retrieves repository information and image metadata required for further analysis.

The **Image Collector module** is responsible for collecting Docker images and related information using Docker Hub APIs. It gathers details such as repository names, image tags, pull counts, layer information, and configuration files. This metadata provides important insights into the structure and dependencies of Docker images.

After collecting the data, the information is stored in the **system database** for further processing. The collected dataset is then used by other modules such as vulnerability scanning, secret detection, and risk evaluation to perform security analysis on Docker images. This structured data collection process ensures accurate and efficient analysis of the Docker image ecosystem.

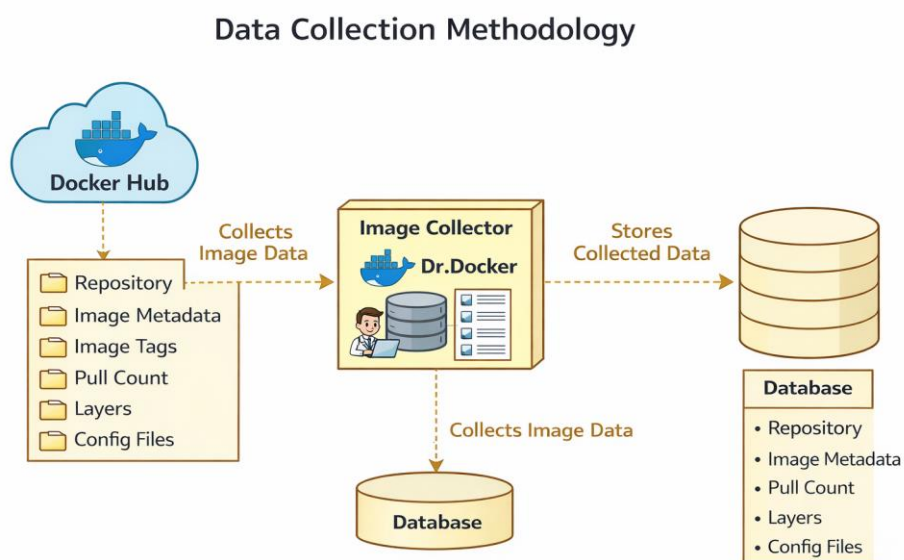


Figure: Data Collection Methodology

3.5 DOCKER IMAGE DEPENDENCY ANALYSIS

Docker images are commonly built using other existing images known as base images, creating a layered structure. Each layer in a Docker image represents a specific change or instruction added during the image creation process. Because of this layered architecture, many images depend on other images, forming a network of dependencies within the Docker ecosystem.

The dependency analysis module examines these relationships by identifying parent and child image connections. When a new image is created from a base image, it inherits all packages, configurations, and libraries present in that base image. As a result, security vulnerabilities present in a base image can propagate to multiple dependent images.

By constructing a dependency graph, the Dr.Docker system identifies influential images that have many dependent images. Analyzing these dependencies helps detect security risks that may spread across the Docker ecosystem and enables more effective vulnerability tracking and risk assessment.

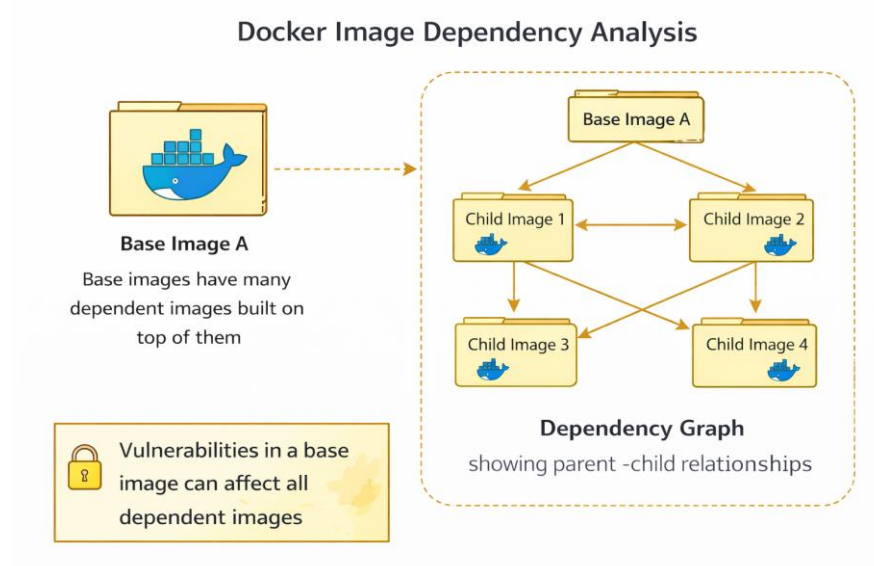


Figure: Docker Image Dependency Analysis

3.6 THREAT DETECTION FRAMEWORK

The threat detection framework in the Dr.Docker system is designed to identify potential security risks present in Docker images before they are deployed. This framework analyzes Docker images using multiple security scanning techniques to detect vulnerabilities, exposed secrets, misconfigurations, and malicious components. By performing automated threat detection, the system ensures that insecure images are identified early in the analysis process.

The framework works by scanning the contents and configuration of Docker images using specialized modules such as vulnerability scanners and secret detection tools. These modules examine installed packages, configuration files, and image layers to identify security issues that may compromise container environments.

After detecting possible threats, the system evaluates the severity of each issue and sends the results to the Risk Engine for further analysis. The calculated risk score helps determine whether the container image can be safely deployed or requires restrictions. This framework provides a structured approach for identifying and managing security risks within the Docker image ecosystem.

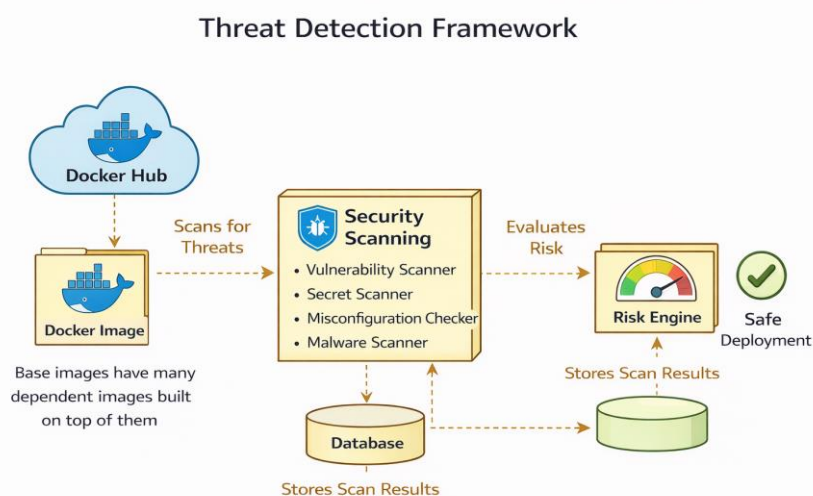


Figure:Threat Detection Framework

3.6.1 SENSITIVE COMMAND PARAMETER DETECTION

Sensitive command parameters in Docker containers may weaken container isolation and introduce security risks. The proposed system analyzes container runtime commands and configuration parameters to identify unsafe options that may allow privileged access or bypass security restrictions. Detecting such parameters helps prevent insecure container execution and improves system security.

3.6.2 SECRET LEAKAGE DETECTION

Docker images may accidentally contain sensitive information such as API keys, passwords, authentication tokens, or private keys. The system scans image layers and configuration files to identify exposed secrets that could lead to unauthorized access. Early detection of secret leakage helps protect confidential information and reduces security vulnerabilities.

3.6.3 SOFTWARE VULNERABILITY DETECTION

Many Docker images include software packages that may contain known vulnerabilities. The proposed system analyzes installed packages and dependencies within the image to identify potential security flaws. Detecting these vulnerabilities helps developers understand the risk level of the image and take necessary actions before deployment.

3.6.4 MISCONFIGURATION DETECTION

Improper system configurations may expose Docker containers to security threats. The system examines configuration files and container settings to identify insecure configurations such as weak permissions or exposed services. Detecting misconfigurations helps improve container security and prevents potential attacks.

3.6.5 MALICIOUS FILE DETECTION

Some Docker images may contain suspicious or malicious files that can compromise system security. The system analyzes the contents of Docker image layers to detect unusual scripts, binaries, or unauthorized software. Identifying malicious files helps ensure that only safe and trusted images are deployed in the container environment.

3.7 TOOLS AND TECHNOLOGIES USED:

DOCKER PLATFORM:

Docker is the containerization platform used to create, manage, and run container images. In the proposed system, Docker images are analyzed to identify potential security risks before deployment. Docker provides the layered architecture that allows the system to inspect image layers and configuration data.

DOCKER HUB:

Docker Hub serves as the primary source of container images used for analysis. It is a public container registry where developers publish and share Docker images. The Dr.Docker system retrieves repository metadata and image information from Docker Hub for security analysis.

PROGRAMMING ENVIRONMENT:

The proposed system can be implemented using a programming environment that supports automation and data processing. Programming frameworks help develop modules responsible for data collection, vulnerability scanning, and risk evaluation.

DATABASE SYSTEM:

A database system is used to store collected image metadata, security analysis results, and calculated risk scores. The database helps maintain structured records of Docker

images and their associated security information.

SECURITY ANALYSIS TECHNIQUES:

Security scanning techniques are used to detect vulnerabilities, secret leakage, misconfigurations, and malicious files in Docker images. These techniques help identify potential security threats before containers are deployed.

VISUALIZATION TOOLS:

Visualization tools are used to present the analysis results in an understandable format. Charts, reports, and tables help developers and administrators easily interpret the security status of Docker images.

3.8 SECURITY ANALYSIS AND METRIC GENERATION

The security analysis and metric generation module evaluates the security condition of Docker images based on the threats detected during the scanning process. After the system identifies vulnerabilities, secret leaks, misconfigurations, and malicious files, the collected information is analyzed to determine the overall risk level of each image.

The system calculates security metrics by considering different factors such as the number of detected vulnerabilities, the severity level of each issue, and the presence of exposed sensitive information. These metrics help quantify the security risks associated with Docker images and allow the system to classify images into different risk categories.

The generated metrics are then used to compare Docker images and identify high-risk containers that may introduce security threats in deployment environments. By analyzing these metrics, the system helps developers and administrators make informed decisions when selecting Docker images for secure deployment.

3.9 WORK FLOW OF THE PROPOSED SYSTEM.

The workflow of the proposed **Dr.Docker system** describes the sequence of steps involved in analyzing Docker images and identifying potential security threats before deployment. The process follows a structured approach that ensures Docker images are properly evaluated before they are used in containerized environments.

The workflow begins with **data collection**, where the system retrieves Docker image information and repository metadata from **Docker Hub**. The Image Collector module gathers details such as repository names, image tags, configuration files, and layer information. This data forms the basis for further analysis of Docker images.

After collecting the images, the system performs **dependency analysis** to identify relationships between Docker images. Since Docker images are built using base images, vulnerabilities in a base image may propagate to other dependent images. The dependency analysis module helps detect these relationships and identify critical images within the ecosystem.

Next, the system performs **threat detection** by scanning Docker images to identify security issues such as vulnerabilities, secret leakage, misconfigurations, and malicious files. This stage helps detect potential risks that may compromise container security. Following threat detection, the system conducts **security analysis and metric generation**, where detected issues are evaluated and security metrics such as vulnerability counts and risk scores are calculated. These metrics help determine the overall security level of Docker images.

Finally, the analysis results are presented through the **result visualization and reporting module**, which generates reports that help developers understand the security condition of Docker images and decide whether the container can be safely deployed or should be restricted.